Structured KV Cache Quantization for Mobile NPUs

Jeongwoo Han
Department of Digital Media
and Communications Engineering
Sungkyunkwan University
jwhot.han@skku.edu

Dongkun Shin*
Department of Electrical
and Computer Engineering
Sungkyunkwan University
dongkun@skku.edu

***Abstract*—Key–value (KV) cache quantization reduces storage, but encoding and decoding overhead can limit its benefits on mobile neural processing units. We present a hardware-oriented implementation of Structured K8, a two-region piecewise-uniform 8-bit key quantizer with arithmetic indexing and lookup-free Hexagon Vector eXtensions (HVX) decoding. We integrate it into a mixed-precision KV cache and compare it with Lloyd–Max under matched storage budgets with Odd and Boundary layer placements. We evaluate Qwen3-1.7B (W4A16) on a Galaxy S26 Ultra at a context capacity of 1024 tokens. Across ten warmed runs per configuration, Structured K8 reduces mean time to first token by 31.12–32.62% and increases mean autoregressive decoding throughput by 13.61–14.41%. On four reused WikiText-2 reference windows, perplexity is 1.81% higher under Odd placement and 0.45% lower under Boundary placement. A separate four-document held-out comparison yields 11.28% lower perplexity than a uniform quantizer, with similar observed throughput. These results demonstrate reduced codec overhead with placement-dependent quality in the evaluated mobile setting.**

***Index Terms*—KV cache, quantization, mobile NPU, piecewise-uniform quantization, Hexagon HVX**

I. Introduction

Key–value (KV) cache quantization reduces storage and data movement, but encoding and decoding overhead can limit its benefits for mobile inference. KIVI motivates asymmetric key/value quantization [1], while TurboQuant explores rotation-based scalar quantization [2]. Practical deployment therefore requires evaluating codec costs alongside quantization error.

Our work builds on a TurboQuant implementation in the Qualcomm AI Hub Models pipeline. Its inherited Lloyd–Max encoder uses comparison trees and reconstruction-level selection subgraphs, while LUT-based decoding requires table lookup and layout conversion. We investigate whether structured reconstruction levels can simplify both paths while maintaining useful language-model quality.

Building on piecewise quantization approaches such as PWLQ [3] and PTQ4ViT [4], we present Structured K8, a two-region piecewise-uniform key quantizer with arithmetic indexing and HVX decoding. Our contribution is its hardware-oriented implementation for a rotated KV cache, preserving the existing byte format and explicit FP16 rounding, together with controlled on-device comparisons against Lloyd–Max K8 and a single-spacing uniform baseline. Mixed key/value precision and layer placement support this implementation. Figure 1 summarizes the overall design.

*Corresponding author: Dongkun Shin.

II. Background and Motivation

A. Key and Value Quantization Errors

Scaled dot-product attention is [5]

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \tag{1}$$

where Q, K, and V are queries, keys, and values, and d is the key dimension; causal masking is omitted. Key quantization changes the dot products and thus the normalized exponential softmax weights. In contrast, value errors enter the weighted sum linearly for fixed Q and K. This motivates asymmetric K/V precision, without implying that key errors always dominate.

B. Layer-Wise Quantization Sensitivity

KV quantization sensitivity varies across layers [6], motivating higher precision in selected layers under a fixed cache budget. We mix K8 and K4 to maintain average K6 precision, with V2 throughout. Odd and Boundary are tested placements, not presumed optima; comparisons use matched masks.

C. Packing and Scalar-Index Costs

Two- and four-bit codes divide a byte evenly, whereas eight-bit codes use direct byte storage. Three-, five-, six-, and seven-bit codes may cross byte boundaries and need fragment assembly. Separately, the inherited Lloyd–Max encoder incurs bit-width-dependent comparison-tree and reconstruction-level selection costs. Structured K8 uses byte storage and arithmetic indexing and decoding; K4/V2 retain packing overhead. Thus, nominal bit width alone does not determine execution cost.

III. Structured Quantization and Implementation

A. Overview

The three panels in Fig. 1 connect the storage budget to the deployed codecs. In the left panel, B_K and B_V denote key and value index-storage budgets, so $B_{KV} = B_K + B_V$ excludes

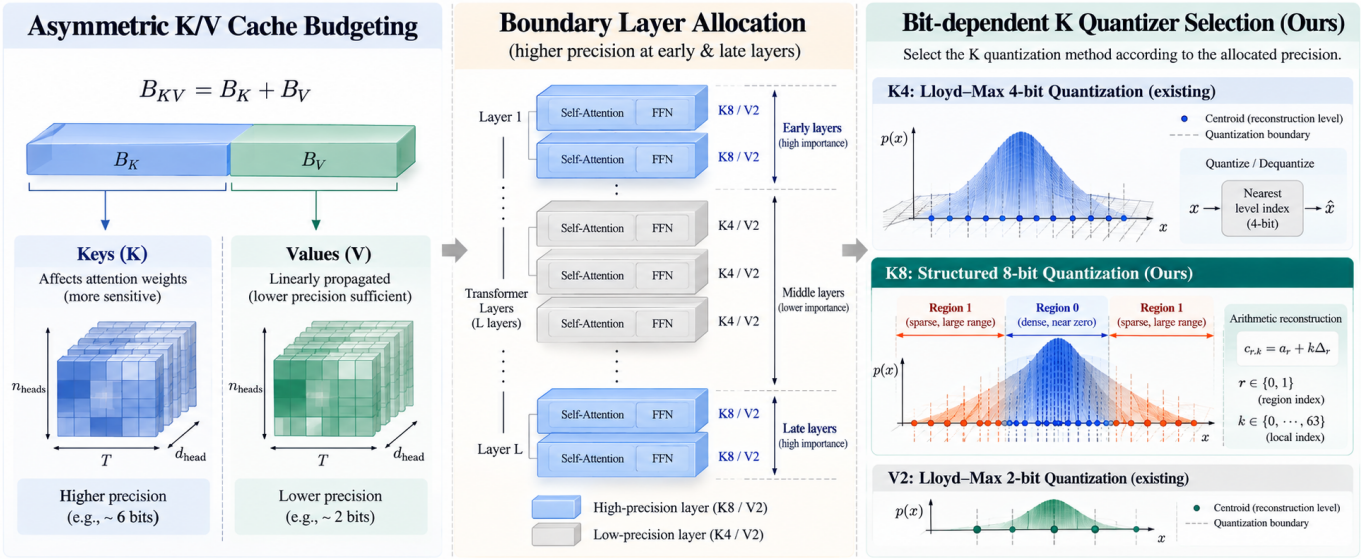


Fig. 1. Conceptual overview of the mixed-precision KV cache, illustrated with Boundary placement. Left: asymmetric key/value index budgeting. Center: layerwise K4/K8 allocation. Right: Lloyd–Max K4/V2 and Structured K8 quantization. Distributions, block proportions, and level markers are schematic; K4, K8, and V2 use 16, 256, and 4 signed reconstruction levels, respectively. Sensitivity and importance labels express the design motivation rather than a measured universal ranking.

FP16 scales. The cache dimensions are token count T, KV-head count n_{heads}, and head dimension d_{head}. Assigning K8 to half of the layers and K4 to the rest gives an average key precision of six bits; values use V2 throughout.

The center panel illustrates Boundary placement, with K8 assigned to the first and last seven of 28 layers. The drawing numbers layers from 1 to L; our experimental masks use zero-based indices. Odd placement is also evaluated under the same storage budget. These placements instantiate the precision allocation used to compare quantizer implementations.

The right panel selects Lloyd–Max for K4/V2 and Structured quantization for K8. Its scalar axis x represents a normalized, rotated coordinate, denoted y below, and $c_{r,k}$ denotes a positive reconstruction level. The following subsections specify arithmetic indexing and native HVX decoding.

B. Quantization Levels and Arithmetic Indexing

The proposed key cache uses Lloyd–Max K4 and Structured K8. For bit width p, the positive reconstruction levels are

$$\ell_p^+(j) = \begin{cases} \mathcal{L}_4^+[j], & p = 4, \\ a_r + k\Delta_r, & p = 8. \end{cases} \tag{2}$$

For K4, $\mathcal{L}_4^+$ contains eight positive Lloyd–Max levels indexed by $j \in \{0, \ldots, 7\}$; the threshold-based encoder is retained. For K8, $j = 64r + k$, with region $r \in \{0, 1\}$, local index $k \in \{0, \ldots, 63\}$, offset a_r, and spacing Δ_r. Negative levels follow by sign reflection. The K8 expression describes the grid before rounding; its multiplication and addition are separately rounded to FP16.

The four coefficients are fitted offline to minimize scalar MSE under the Gaussian approximation $\mathcal{N}(0, 1/d)$, without KV samples or PPL-based tuning.

Arithmetic indexing. For a normalized, rotated coordinate y, let $m = |y|$. The boundary τ is the midpoint between the FP16-rounded levels at $(r, k) = (0, 63)$ and $(1, 0)$. Select the region as

$$r = \begin{cases} 0, & m \le \tau, \\ 1, & m > \tau. \end{cases} \tag{3}$$

The local index is

$$k = \text{clip}\left(\left\lfloor (m - a_r)\frac{1}{\Delta_r} + 0.5 \right\rfloor, 0, 63 \right) \tag{4}$$

The logical floor rule rounds half-step ties upward. Execution uses a stored reciprocal with intermediate FP16 rounding, which can shift decisions near index boundaries.

Byte encoding. The magnitude index j and the sign of y are stored in one UINT8 code:

$$b = \begin{cases} 128 + j, & y \ge 0, \\ 127 - j, & y < 0. \end{cases} \tag{5}$$

For negative values, the magnitude index is reflected to preserve compatibility with the existing ascending LUT layout. The decoder recovers $j = b - 128$ for $b \ge 128$ and $j = 127 - b$ otherwise.

C. HVX Arithmetic Decoder

The native HVX decoder recovers r and k from each stored byte and reconstructs $a_r + k\Delta_r$ using separate FP16 multiply and add operations. It then applies the sign and the effective FP16 scale used for norm correction, producing FP16 tiles for the existing attention path. Vectorized decoding avoids level-table loads and per-coordinate scalar loops. The coefficient

payload is four FP16 values (8 bytes), replacing a 256-entry FP16 codebook (512 bytes).

IV. Experimental Methodology

Experiments use Qwen3-1.7B [7] (W4A16) on a Galaxy S26 Ultra (SM8850), with QAIRT 2.48.0.260626 and the HTP backend in burst mode. Inference uses a single sequence and context capacity of 1024. Structured K8 and Lloyd–Max K8 are compared under matched masks in Odd and Boundary placement. Weights, activation encodings, rotations, norm correction, and K4/V2 paths remain fixed. The quantized Johnson–Lindenstrauss (QJL) residual stage is disabled, so the evaluated codec is not the full TurboQuant construction [2].

Performance uses 897 prompt tokens and 128 generated tokens. Each invocation includes a full warm-up, a cache reset, and a measured session. TTFT excludes model loading; decode throughput is 127 divided by the total duration of the post-prefill autoregressive steps. Each configuration has ten measured sessions in counterbalanced order, with a ten-second pause between invocations. We report means and sample standard deviations, with improvements computed from group means. End-to-end timing excludes detailed profiling.

Quality is evaluated by teacher-forced perplexity on four reused WikiText-2 [8] reference windows of 1024 tokens, using the unchanged performance binaries. Each window is scored once per configuration, totaling 4092 target tokens per configuration. Aggregate perplexity is the exponential of the token-averaged negative log-likelihood (NLL) across all four windows. These windows are neither an independent held-out test nor a full-corpus evaluation. Earlier Lloyd–Max and uniform held-out results use different document sets and are reported separately; their absolute PPL values are not directly comparable across sets. No matched Boundary held-out comparison is available. The three-run uniform timings and other supporting experiments are not pooled with the repeated Lloyd–Max measurements.

V. Results and Analysis

A. End-to-End Performance and Quality

We compare Structured K8 with Lloyd–Max K8, which has lower Gaussian scalar reconstruction MSE. This comparison evaluates complete implementations.

Under Boundary placement, Fig. 2 shows that Structured reduces mean TTFT by 31.12% and increases prefill and decode throughput by 45.22% and 14.41%, respectively. Separate Odd-placement runs show a 32.62% TTFT reduction and 48.49% and 13.61% higher prefill and decode throughput, respectively. The speed advantage is observed under both tested masks on this device.

On the four reused reference windows (Table I), Boundary lowers PPL by 12.65% for Lloyd–Max and 14.58% for Structured relative to Odd. Structured's PPL is 1.81% higher than Lloyd–Max under Odd and 0.45% lower under Boundary. Lloyd–Max wins three Odd windows; each codec wins two Boundary windows.

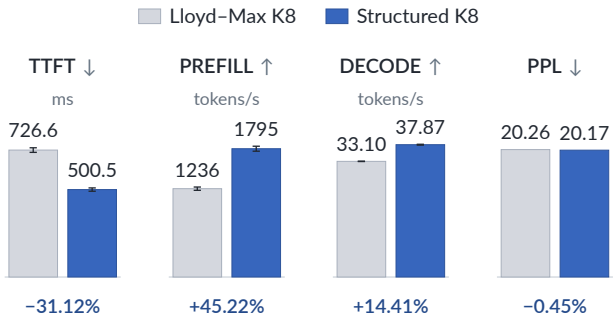


Fig. 2. Boundary-placement performance and reference-window PPL. Timing: mean ± sample SD over 10 runs per configuration, with 897 prompt and 128 generated tokens. Percentages show changes from Lloyd–Max K8.

TABLE I
Reference-window PPL reduction from Odd to Boundary placement.

K8 codec	Odd PPL ↓	Boundary PPL ↓	Reduction (%)
Lloyd–Max	23.19	**20.26**	12.65
Structured	23.61	**20.17**	14.58

B. Comparison with Uniform Quantization

The uniform baseline uses a symmetric 256-level grid with one spacing fitted under the same Gaussian model, without KV- or PPL-based tuning, and frozen to FP16. Both variants use arithmetic HVX decoders and the same Odd placement, allowing a comparison of single- and two-region quantization under matched deployment settings.

In Fig. 3, Structured reduces PPL by 11.28% on the separate four-document held-out set B. The three-run timings have small differences and overlapping ranges, providing no clear speed advantage for either variant. Structured improves NLL on three documents, but one accounts for 79.65% of the net gain. The quality benefit is therefore limited evidence from this small set. Reference-window PPL is 23.7939 for Uniform and 23.6122 for Structured.

C. Encoder and Decoder Analysis

The standalone Structured K8 encoder has 76 ONNX nodes, compared with 837 in the inherited Lloyd–Max K8 encoder, mainly through simpler scalar indexing and level/scale reconstruction. In the single AR128 probe using the same captured layer-0 input (Table II), SDK execute time decreases from 3187 to 1523 μs. These observations support the reduced implementation cost.

For KV-cache decoding, Table III compares the Structured LUT control and arithmetic decoder using identical codes, scales, and reconstruction levels in a single 256-token probe. Both produce outputs bit-exact against the FP16 reference. Arithmetic reduces physical HVX busy cycles by 39.56%, while SDK execute time decreases only from 4469 to 4429 μs (about 0.9%). The cycle reduction measures vector-unit work, not whole-model decoding speedup. Additional

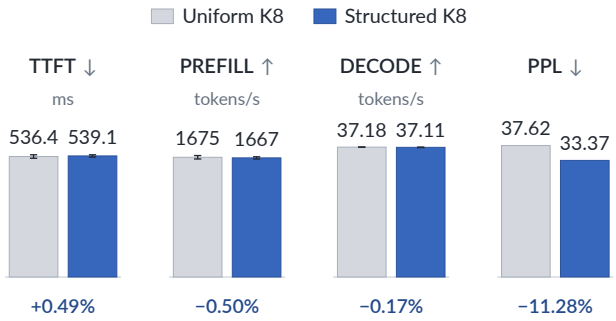


Fig. 3. Odd-placement performance and held-out PPL (set B). Timing: mean ± sample SD over three runs per configuration, with 897 prompt and 128 generated tokens. Percentages show changes from Uniform K8.

TABLE II
AR128 ENCODER PROBE ON THE SAME CAPTURED INPUT. NAMED-OP CYCLE COUNTERS ARE NOT DISJOINT PHYSICAL OCCUPANCY OR ELAPSED TIME. EACH RESULT IS A SINGLE INSTRUMENTED PROBE.

Metric	Lloyd–Max	Structured
Scalar-index cycles	15,666,367	1,146,338
Norm/scale cycles	12,195,259	822,695
SDK execute (μs)	3,187	1,523

TABLE III
SAME-CODEBOOK STRUCTURED DECODER PROBE: ONE INSTRUMENTED 256-TOKEN INVOCATION PER VARIANT.

Metric	LUT control	Arithmetic
Physical HVX busy cycles	593,957	358,972
SDK execute (μs)	4,469	4,429
Execute excluding wait (μs)	769	653

decoder checks cover all byte codes, edge scales, and partial tiles.

VI. CONCLUSION

Structured K8 uses two-region reconstruction levels to simplify arithmetic indexing and HVX decoding for mobile KV caching. Under Odd and Boundary placement, it reduces mean TTFT by approximately 31–33% and increases mean decode throughput by about 14% relative to Lloyd–Max K8. PPL ordering depends on placement within the evaluated reference windows. Future work should extend evaluation to larger independent datasets, additional models and devices, and longer-context workloads.

REFERENCES

[1] Z. Liu *et al.*, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," arXiv:2402.02750, 2024.
[2] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "Turbo-Quant: Online vector quantization with near-optimal distortion rate," arXiv:2504.19874, 2025.
[3] J. Fang, A. Shafiee, H. Abdel-Aziz, D. Thorsley, G. Georgiadis, and J. Hassoun, "Post-training piecewise linear quantization for deep neural networks," arXiv:2002.00104, 2020.
[4] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, "PTQ4ViT: Post-training quantization for vision transformers with twin uniform quantization," in *Proc. ECCV*, 2022.
[5] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[6] X. Li *et al.*, "KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference," in *Proc. ICML*, PMLR, vol. 267, pp. 36451–36485, 2025.
[7] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.
[8] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," arXiv:1609.07843, 2016.